Rogue AI Detection in the Wild

Findings, challenges and insights on detecting and preventing rogue AI agent activity

Whitepaper | 25 October 2026 | Companion to the methods report and the Python prototype. v6 and Phases 1 to 3 added; v10 collusion-carrier section (6.1) added 25 October 2026; the 23 October 2026 text stands where it is not revised here.

Data notice: the prototype described in this paper runs only on synthetic data generated by the project. No real platform, account or person was scraped or analyzed. Prototype results show that the pipeline works; they are not evidence of real-world detection accuracy. A later loopback sample (section 4) drives a local model and Playwright against 127.0.0.1 only. That sample is not wild traffic and is not a benchmark. The web honeypot does not auto-block visitors. A lab gateway can freeze only a registered internal key.

1. Executive summary

AI agents that act through tools and APIs can be hijacked, misused or deliberately deployed for abuse. Detecting this from outside the systems an organization controls is hard, and the published evidence says no single technique is reliable. This paper summarizes what a review of public sources supports, shows how a small prototype puts those ideas together, and sets out what should be done to prevent rogue AI activity rather than only detect it.

- Layer the defenses. Unsupervised behavioral scoring, graph-based coordination analysis and tool-call monitoring each fail in different ways. Combining them, with human review, performed far better than the Isolation Forest alone in the prototype's synthetic test.
- Score behavior, not intent. Intent cannot be observed. Every flag is a lead for a person, and unsupervised evidence on its own should be capped at low severity until corroborated.
- Prevention beats detection. OWASP and MITRE ATLAS both frame agent harm around tool use, permissions and autonomy. Least privilege, approval for high-impact actions and complete logging reduce what a rogue agent can do and make misuse visible [4][5][7].
- Evasion is easy. The prototype's evasive variants dropped fused recall from 1.00 to 0.11 on synthetic data. Published work also shows that LLM-based detectors can be attacked [10].
- Synthetic scores overstate a small local model. On archived loopback runs of qwen2.5:3b against the v6 honeypot, 2 of 13 non-control sessions were caught. The model usually stopped after 1 to 3 tool calls, so deeper traps (the decoy MCP path, the coordination board, provenance cursors) rarely had a request to score. The web honeypot still only raises a lead for a person. See section 4.
- Collection is bounded by law and ethics. US case law has narrowed the CFAA, but contract terms, data protection law and platform rules still apply, so building on synthetic or pseudonymized data should be the default [14][15][16][17].

2. Problem and scope

There is no standard definition of a rogue AI agent. This work uses a working one: an automated, model-driven or model-assisted actor that pursues actions outside its operator's intended or authorized scope, or is operated to cause harm, on systems the detector's owner does not control. It covers two cases that look the same from outside: an agent hijacked through injected instructions, and an agent deliberately deployed for abuse.

The project brief names six components: data collection, anomaly detection, real-time monitoring with alerting, collaboration and intelligence sharing, an ethical and compliance framework including responsible disclosure, and a community engagement platform. The prototype implements each at small scale; the report covers the underlying methods and sources in detail.

3. What the evidence says

3.1 Bot and agent detection

Social bot detection is the most mature neighboring field. Botometer evolved from BotOrNot (2016) toward ensembles of specialized classifiers [8]. Yang et al. found that choosing a subset of training data generalized better across datasets than training on everything [9]. The TwiBot-22 benchmark is a graph with 92,932,326 nodes and 170,185,937 edges [11]. Most of this work targets accounts on one platform; an agent acting through APIs has no public profile to classify. Chan et al. propose agent identifiers, real-time monitoring and activity logs, all of which depend on the deployer's cooperation [12].

LLM-based detectors are an attack surface. Orenstein and Birman report accuracy drops of up to 48 percent from adversarial rewriting and prompt injection against LLM bot detectors, and about 86 percent accuracy for a multi-LLM ensemble under strong attack, on 2,000 TwiBot-20 accounts with three open-source models [10]. This is one group's result on one dataset.

3.2 Anomaly and coordination detection

Isolation Forest isolates anomalies with random splits and is a sensible first-pass detector for per-agent features [13]. Pacheco et al. build coordination networks from shared behavioral traces and link accounts only when their behavior is more similar than chance; they stress that coordination does not imply automation [6]. Meta's definition of coordinated inauthentic behavior is likewise behavior-based rather than content-based [18]. For agent-to-agent collusion, Motwani et al. show that steganographic coordination can evade monitoring and that simple mitigations may be insufficient [19].

3.3 Prompt injection and agent misuse

Greshake et al. demonstrated that instructions placed in retrieved data can steer LLM-integrated applications, with impacts including data theft and code execution [3]. OWASP lists Prompt Injection (LLM01) and Excessive Agency (LLM06) and names excessive functionality, permissions and autonomy as root causes [4]; its agentic list puts Agent Goal Hijack first [5]. NIST treats agent hijacking as a form of indirect prompt injection [20]. Anthropic reported in November 2025 that it disrupted an espionage campaign in which its coding tool performed much of the tactical work; this is the vendor's own account and was not independently verified here [21].

3.4 Standards and frameworks

Framework	Role in this work
MITRE ATLAS [1]	Shared vocabulary; alerts carry approximate technique tags (checked against the v2026.09 data file)
NIST AI RMF and AI 600-1 [2]	Governance functions (govern, map, measure, manage) and the generative AI profile
OWASP LLM and Agentic Top 10 [4][5]	Misuse patterns and mitigations
STIX 2.1 and TAXII 2.1 [22]	Structured sharing of indicators; the prototype exports STIX

ISO/IEC 29147 and CERT/CC CVD [23]	Template for a disclosure process
EU AI Act Article 73 [24]	Serious-incident reporting for high-risk AI providers; Annex III rules apply from 2 December 2027 after the Digital Omnibus

4. The prototype

The prototype is a Python application with a command line and a local web dashboard. It uses only synthetic events generated by the project; its network collector is intentionally disabled. It implements:

- A collector interface with a synthetic generator, a validated and PII-redacting importer for community-contributed files, and the disabled live stub.
- Per-agent feature extraction (timing, access, tool use, content and network features).
- Isolation Forest scoring with per-agent explanations, transparent rules keyed to technical gates (denied or out-of-scope requests, new external-destination tool use after reading external content, encoded payloads), and a coordination graph combining timing synchrony, rare-template content similarity and reciprocal messaging.
- A hash-chained alert log with severity, explanation and approximate ATLAS tags; a sliding-window monitor; flagged-case evidence packages; STIX 2.1 export; a community report form; and developer and disclosure documents.

The synthetic data plants three rogue roles (an access prober, a five-agent collusion ring and two injection-driven hijacks) among 53 benign agents, including look-alikes that should not be escalated: scheduled jobs, a heavy crawler, an orchestrator with workers, and routine users of an external tool. The roles are planted by the same author as the detectors, so the results below are a regression check, not a benchmark.

Synthetic run (seed 7, 62 agents, 9 rogue)	**Precision**	**Recall**	**Notes**
Fused pipeline, standard scenarios	1.00	1.00	No benign look-alike escalated above info or low
Isolation Forest alone, standard	0.62	0.56	Flags scheduled jobs and the heavy crawler; misses most of the collusion ring
Fused pipeline, stealth variant	1.00	0.11	1 of 9 found; slow probing, de-synchronized ring and a routine-tool hijack evade
Isolation Forest alone, stealth	0.25	0.22	Mostly false positives

Test status of the first version: 42 automated tests passed (Python 3.13.5, scikit-learn 1.9.1, numpy 2.5.4, networkx 3.7), covering schema and collectors, features, detectors, the pipeline on several seeds, alert-log tamper detection, the STIX export, report validation and the web endpoints. The documented install and run steps were also checked in a clean virtual environment.

Later versions (v3 to v5). The companion code grew into two projects. agent-watch is a localhost-only honeypot and log detector; rogue-ai-pipeline is the multi-layer scorer. v3 added a mirrored access stream, passive web detectors (asset starvation, unlinked jumps), score floors and a decoy MCP endpoint. v4

separated selective asset blocking from true starvation, added an ARIA canary and a three-boolean interaction beacon for browser-backed mimics, a two-signal corroboration gate, per-layer attribution of benign leads, an empirical link-graph fallback and an encoded-secret scrubber port. v5 adds a deterministic harness fingerprint (six archetypes, each with reasons and a low, medium or high confidence that describes how much of the signature was seen) and a static, escaped HTML replay page for human reviewers. On 23 October 2026 the recorded test status was agent-watch 446 passed and 1 skipped, rogue-ai-pipeline 331 passed, pipeline type checks clean. These are regression checks on synthetic data. v6 and the later packages are below.

v6 traps (synthetic regression)

v6, in both agent-watch and rogue-ai-pipeline, adds a challenge-bound browser beacon, a DOM-visual inversion trap, load-bearing provenance cursors, an R14 check for a parasitic session fork, and a static-refusal decoy for an LLM completion endpoint. The honeypot still binds to 127.0.0.1 by default and makes no outbound request. Every web lead stays a lead for human review. Nothing in the honeypot auto-blocks a visitor. The v6 write-up records agent-watch at 492 passed and 1 skipped, and the pipeline at 384 passed, with pipeline type checks clean [46]. Those counts are the regression run recorded with that write-up. This paper does not re-count the full suite.

The paired synthetic check uses clients written by the same author as the detectors. It is a regression check, not a benchmark. Headline contrasts, same traffic scored by the v5 and v6 trees [46]:

Paired synthetic check (same author)	v5	v6
agent-watch human alerts (five human cohorts)	15 of 75	0 of 75
Pipeline web false-positive rate	39 of 575 (0.068)	14 of 575 (0.024)
Pipeline core standard false-positive rate	21 of 605 (0.035)	21 of 605 (0.035)
All-evasive core recall (three original scenarios)	0 of 80	12 of 80
Pipeline leads, stealth browser and header forger	0 of 15 each	10 of 15 and 10 of 15
Pipeline leads, mimic_assets and mimic_linked_nosec	15 of 15 each	0 of 15 each (regression)

The 12 of 80 all-evasive core detections are the access-prober cohort. Evasive exfiltration and the evasive collusion ring stay at 0. The two pipeline regressions are deliberate: in the v6 evidence those mimics look like the beacon-blocked and no-asset humans whose false positives v6 removed. agent-watch still labels mimic_assets as an agent in that simulation, by the beacon and the inversion trap. Catching a client the author wrote is expected. It is not evidence about a model the author did not write.

Loopback Tier-2 sample (Phase 1)

Phase 1 ran the v6 honeypot on 127.0.0.1 with a local Ollama model, qwen2.5:3b (Q4_K_M), and with Playwright Chromium. There was no public deploy. The archive holds 14 scrubbed runs: 9 urllib tool-calling

sessions (three prompts, three seeds), 4 Playwright agent or crawler runs, and 1 scripted human control. "Caught" here means an agent-watch severity of medium or above, or a pipeline lead. On the 13 archives that are not the human control, the catch rate is 2 of 13. The two catches are the headless Playwright crawler and one of the two LLM-driven Playwright runs. The nine urllib sessions were not caught. The scripted human control was not caught either, and it is not in the denominator. A second, larger local model was not run. That skip was about 5 GiB free RAM and no swap on the machine, not a finished multimodel result.

Archived Tier-2 slice (loopback, qwen2.5:3b)	Caught	What the sessions did
urllib tool-calling (9 runs)	0 of 9	Usually 1 to 3 fetch calls, then a stop. Benign, aggressive-audit and stealth prompts.
Playwright crawler and LLM browser (4 runs)	2 of 4	Headless crawler caught. One LLM browser caught on the inversion link; one LLM browser and one stealth-ish run not caught.
Non-control archives together	2 of 13	Report this row for Tier 2. Do not pool it with the synthetic recall above.
Scripted human control (1 run)	0 of 1	A scripted proxy, not a real person. Soft unlinked-jump and cursor advisories only. Excluded from the 2 of 13.

Why the synthetic optimism does not show up here. The simulated aggressive client walks a long list of decoys. qwen2.5:3b usually quit after 1 to 3 steps. It fetched the front page, llms.txt or the OpenAPI document and answered. It rarely reached /mcp, the coordination board, or a provenance cursor, so those traps had nothing to fire on. A 3B CPU model is not a frontier computer-use agent. The prompts, decoys and detectors share an author lineage, and every request hit one loopback honeypot. The sample is too small to call an accuracy.

Two calibrations followed the first pass, because that pass produced false structure, not because the catch rate was disappointing [46]. R14 had treated the single-client Playwright crawler's own fast navigations, beside its own beacon, as a parasitic fork. Same-session R14 now requires a schema-provenance cursor on the rider (and interleaved depth on two disjoint subtrees). A bare unlinked jump from one client does not qualify. Re-analysis of the archived crawler no longer raises parasitic_session_fork. The crawler remains a lead on the other signals (agent-watch critical; pipeline rules R10 and R11). Separately, dwell under 12 ms is inconclusive for touch and for an operating-system tap: the beacon returns null rather than a failed human kinematic. Null does not corroborate the visual-language-model rule. A real phone was not tested. A scripted mouse control, with a longer press, was rechecked at 2 of 2 beacons with dwell marked true.

Playwright archives are the useful half of this sample for offline work. They record navigation, assets and beacons. The urllib traces are thin: a handful of tool calls and a stop. An offline rule needs a path, a referrer or a cursor in the log. The thin traces often have none of the deep-trap paths.

Offline analyzer (Phase 3)

agentwatch-analyzer is a standalone command-line package. It reads static access logs the operator already has. It does not import the honeypot, bind a port, or make an outbound call. It scrubs, then writes a findings file, a static replay.html page, and a STIX 2.1 bundle. The package ships 12 Sigma rules and records 10 of 10 unit tests passed [46]. It is not a web application firewall, and it is not a substitute for the live honeypot.

Offline coverage is incomplete on purpose. Rules that need a beacon, a honeypot clock, or the internal gateway cannot be decided from a normal access log:

Rule	Offline?	What a static log can support
R8 asset starvation	Yes	A browser user-agent and HTML without CSS or script fetches.
R9 unlinked jump	Yes	Referer link graph. Stronger if a sitemap or recorded links are supplied.
R10 schema or machine-doc	Partial	Cursor, query, OpenAPI and llms.txt paths. Decoy kinds need the honeypot.
R11 beacon and mimic	No	Needs honeypot beacons, inversion, or visual-language-model fields.
R12 tool hijack	No	Needs agent-gateway or API telemetry.
R13 timing adaptation	No	Needs honeypot timing.
R14 session fork	Partial, advisory	Interleaved disjoint trees only. No human-beacon corroboration offline.

A smoke run on Jordan's machine, against archived Tier-2 logs and the nginx fixture agentlike_nginx.log, wrote replay.html, findings, and a STIX bundle with validator_ok true. The validator can still warn on x_ custom properties and on approximate ATLAS tags. Those warnings are expected with this exporter. Match counts from that smoke, and no others:

Sample	Matches	Rules
Tier-2 aggressive urllib LLM	0	none
Tier-2 Playwright LLM s100	1	R9
Tier-2 Playwright crawler s100	2	R9 + R14 (advisory offline)
Fixture agentlike_nginx.log	3	R8, R9, R10

The offline R14 hit on the crawler is the advisory form. It is not a return of the live R14 false positive removed in the Phase 1 calibration. Live rescoring of that archive does not raise parasitic_session_fork. Contact with a decoy path in a log is still contact, not intent.

What the prototype shows, and does not

- Shows: that independent evidence types can be fused into explained alerts, that benign periodic or orchestrated behavior can be handled explicitly, and that evasions are easy to plant and expected to work.
- Does not show: performance on wild traffic, behavior under drift, or resistance to an attacker who adapts. The loopback sample shows that synthetic recall did not transfer to one small local model. It is not a benchmark. No wild deployment is claimed.

5. Challenges

Challenge	Why it matters
Intent and attribution	Behavior can be anomalous for benign reasons; a flag is not proof, and naming an operator wrongly causes real harm.
Evasion and adaptation	Adversarial rewriting, slow-and-low activity, and abuse of already-granted tools defeat simple detectors.
No external agent identity	Outside a deployer's own systems there is no agreed way to identify an AI agent or obtain its logs [12].
Labeled data	Little public labeled data exists for agent-to-agent misconduct; benchmarks such as TwiBot-22 are social-platform specific and gated [11].
Benign look-alikes	Schedulers, crawlers and orchestrators are unusual or synchronized too. Fan-out workloads triggered by one cause can look like coordination.
Detector drift and attack surface	LLM-based detectors can be attacked and can change as models are updated [10].
Legal and privacy limits	Collection must respect terms, data protection law and platform rules; EU platform data has a formal researcher route [14][15][16][17].
Human workload	Alert fatigue and the cost of review limit how aggressive thresholds can be.

6. Preventing rogue AI activity

Detection from outside is a backstop. The larger gains come from controls on the systems that host agents:

- Least privilege. Give each agent only the tools, data and permissions its task needs; avoid open-ended tools; run actions in the requesting user's context [4].
- Approval for high-impact actions. Require a human or independent check for deletion, external sends and privilege changes [4].
- Treat external content as untrusted. Retrieved pages, files and tool outputs can carry instructions; monitor for goal changes after ingestion [3][5].
- Identity and logging. Assign agent identifiers and keep tamper-evident activity logs so misuse can be attributed and investigated [12].
- Baseline and alert on tool-call sequences. Behavioral deviation (a new external-destination tool after reading external content) is more robust than content filtering.

- Incident process. Prepare detection, investigation, correction and reporting workflows, including obligations such as AI Act Article 73 where they apply [24].
- Share carefully. Exchange indicators in structured form with partners, after review and with personal data removed [22][23].

Lab gateway for registered internal keys (Phase 2)

The controls above are the point of agent-gateway. It is a loopback policy proxy (pep.py) between an organization's own agent harness and JSON-RPC or MCP-style tool calls. The project maps it to MITRE ATLAS mitigation AML.M0030, restrict tool invocation on untrusted data: untrusted ingest taints the session, and a tainted call can be blocked or the key frozen [46]. The mapping is the project's, recorded in the Phase 2 note. This paper did not re-check that mitigation identifier against the ATLAS data file.

The safety split is the whole design. The web honeypot stays human-review only and does not auto-block a public visitor. Freeze and hard-block apply only to API keys listed in the lab registry (config/agents.json). The default bind is 127.0.0.1. Tool backends in this prototype are simulated echoes. There is no outbound call unless a webhook stub is turned on, and that stub defaults to off. This is not a production web application firewall, and it is not Web Bot Auth [34].

The recorded test run is 14 of 14 passed, including a 7 of 7 ATLAS falsification suite: tainted external parameter egress, scratchpad inheritance, a bound approval token that rejects an argument swap and a replay, delegation fan-out, tools/list ingest, and an allowlisted constrained sink [46]. These tests check that the gates reject the cases they were written to reject. They are not a production evaluation, and they say nothing about visitors who were never given a lab key. On the 25 October 2026 tree the full agent-gateway suite is 262 passed (pytest in agent-gateway at commit cefec0d), the same kind of same-author check at a larger size.

6.1 Collusion carriers and the evaluate-collusion check (v10, 25 October 2026)

Agents that share tools can coordinate without putting a secret in any single argument. The v10 gateway adds deterministic checks for three such carriers, with no model call on the decision path [47]:

- Side channels (Group A). side_channel_guard.py bounds what a session that is both tainted and has read sensitive data can still signal: the number of distinct argument variants, the number of released outbound calls, and how regular their timing is. Crossing a bound holds the session for a person and freezes the registered root key for 15 minutes.
- Delegation laundering (Group B). delegation_guard.py treats each hand-off as a lattice meet: the recipient keeps every restriction either side carried, written as chained-HMAC caveats on its agw1. token. A delegated principal that was barred upstream is denied egress in any session (confused_deputy_delegation_blocked).
- Split and encoded smuggling (Group C). deobfuscate.py keeps salted SHA-256 digests of 8-character n-grams of a sensitive value and its Base64, hex, URL and whitespace-stripped forms, matched over a per-session window of released arguments. canary_watermark.py adds a zero-width HMAC watermark bound to session, step and tool, plus credential digests, so a hit names the step it came from.

Supporting v10 modules: arg_ast_guard.py (lexical argument firewall), egress_uri_guard.py (canonical SSRF and cloud-metadata guard), toxic_combinations.py (static 2-hop and 3-hop toxic-flow reachability), sarif_export.py (SARIF 2.1.0 for GitHub code scanning), and audit_chain.py (SHA-256 hash-chained decision log and an HMAC-signed audit manifest).

python -m agent_gateway evaluate-collusion --report runs the repository's own defence tests by group, times benign decisions, and verifies the decision log. The recorded run at commit cefec0d: Group A 8 of 8, Group B 8 of 8, Group C 41 of 41 (57 of 57). The suite holds 5 stateful Hypothesis machines, including a delegation DAG machine at 120 examples of 24 steps. Across 750 benign decisions (5 workflows, 50 runs each) there were 0 denials, and the hash-chained log verified over 750 of 750 entries; an edited or dropped line in a copy was detected. CPU time per decision was p50 0.126 ms, p95 0.892 ms and p99 0.999 ms against a 2 ms reference, with a slowest single decision of 3.84 ms. Timing includes decision-log I/O and is from one machine.

These figures say that the guards behave as their own tests specify. The scenarios and the benign workflows were written by the same author, so the counts are not an estimate of recall against an unknown adversary and the zero denials are a smoke check, not a false-positive rate. They are kept apart from the real-model loopback figure in section 4 (2 of 13 non-control runs), which is a separate measurement and is never pooled with them.

7. Ethics, legal constraints and responsible disclosure

Four sources set the boundaries most relevant to data collection. This is a research summary, not legal advice.

Source	Constraint	Design consequence
Van Buren v. United States (2021) [14]	CFAA 'exceeds authorized access' means entering areas one is barred from, not misusing allowed access	Define unauthorized access by technical gates; never bypass controls
hiQ v. LinkedIn (9th Cir. 2022) [15]	Upheld a preliminary injunction; CFAA likely does not reach public pages; no final merits ruling	Public-page collection has support in that circuit; other claims remain
Meta v. Bright Data (N.D. Cal. 2024) [16]	Contract claim failed on logged-off scraping of public data	Read each platform's terms; logged-in access is governed by them
EDPB Guidelines 03/2026 (draft) [17]	Scraping for generative AI training: minimize, consider synthetic data, exclude sites that oppose scraping	Scope is AI training by private entities; apply by analogy. Prefer synthetic or pseudonymized data

The Menlo Report's four principles (respect for persons, beneficence, justice, and respect for law and public interest) frame the project policy [25]. The project ships a collector compliance checklist and responsible disclosure guidelines adapted from coordinated vulnerability disclosure practice: report privately first, state confidence honestly, give the operator a route to dispute, and publish only aggregate lessons.

8. Recommendations

Near term

- Adopt the collector checklist and keep all development on synthetic or pseudonymized data until legal and ethics review is complete.
- Instrument agents you operate with identifiers, scoped credentials and tool-call logs, and feed them to the detectors.
- Extend the evaluation with adversarial and benign-look-alike cases, and report false positives and review workload alongside detection rates.

Medium term

- Build partnerships with security researchers and academic groups for labeled data and independent validation; use sanctioned data-access routes for platform data.
- Stand up a disclosure program with a published contact, dispute path and partner review before sharing indicators.
- Add drift monitoring and retraining on reviewed windows; test detectors against adaptive attackers.

Open research questions

- An accepted external identity for AI agents.
- Public labeled data for agent-to-agent misconduct.
- Whether communication monitoring can catch steganographic collusion.

9. Related work and comparative architecture

This section compares the companion code with work that already exists: a honeypot and log detector (decoy API documents, a decoy MCP endpoint, canary pages, a coordination board with a handoff token and nonce, a secret scrubber, passive detectors for asset starvation and unlinked jumps, an ARIA canary and a three-boolean interaction beacon, score floors, a harness fingerprint, a static replay page) plus the multi-layer pipeline. The comparison rests on a limited public web search on 23 October 2026, not a systematic review. Each source in the table was opened and read; the full table, dates and the items that could not be verified are in the project file docs/prior-art.md [45]. Wherever the table says “not found in my search”, that means exactly that: I did not find it documented in the pages I read, which is not evidence that the capability is absent. Drosera [27] overlaps the web side most closely. v6 trap details, the loopback sample, the offline analyzer and the lab gateway are in sections 4 and 6. They are not folded into the synthetic counts in the table.

Capability	**Documented in prior art (what I read)**	**This stack (synthetic numbers are same-author; loopback is section 4, not this table)**
Hidden-instruction canary pages	Palisade [26]; Drosera [27]; Thinkst [29]. Documented.	Yes. Contact with a canary is a lead, not proof of intent.
Per-session token / nonce round trip	Drosera: HMAC-signed ticket [27]. Others: not found in my search.	Handoff token plus nonce on the coordination board; a token replayed from another session or address is a separate signal (prior art: not found in my search).
Decoy MCP endpoint, tools/list then call	Sundew [28]; MCP Decoy [31]; HoneyMCP [32]; Thinkst [30]. Documented.	Yes; the harness fingerprint adds an mcp_tool_runner label.
Asset starvation	Drosera: no-assets behavior signal [27]. Others: not found in my search.	Yes. Since v4 it separates selective asset blocking from true starvation: a client that loads CSS or script, or sends coherent Sec-Fetch headers, is not flagged.
Move checked against the links the page actually offered	Not found in my search.	Yes (unlinked jump), with an empirical fallback from Referer transitions when the log does not record offered links. The fallback is circular within one batch.

Capability	Documented in prior art (what I read)	This stack (synthetic numbers are same-author; loopback is section 4, not this table)
Browser-backed mimics	FP-Agent and Fayolle et al. study client-side and multi-layer fingerprints [38][39]. Honeypot traps for this case: not found in my search.	ARIA canary plus a beacon of three booleans (no identifiers). Catches simulated mimics that fetch our script but never run it. JavaScript-disabled and beacon-blocking people produce no beacon: a known false positive.
Harness archetype labels	Model attribution from action traces [40]. Deterministic archetype labels for web agents: not found in my search.	Six archetypes plus unclassified, each with plain-language reasons; descriptive, never scored.
Downstream pipeline	Drosera documents noisy-OR fusion, log replay and a STIX report format [27]. A pipeline combining anomaly, rules, graph, audit chain and STIX: not found in my search.	Isolation Forest, rules, coordination graph, a two-signal corroboration gate, per-layer attribution, hash-chained audit log, STIX 2.1.
TLS and header fingerprints	JA3/JA4 [37]; Fayolle et al. [39]. Documented.	Not implemented (a parse-only field).
Real-world evidence	Palisade: about 8.1 million real attempts [26]; Li et al. [41].	No wild deployment. Loopback only: local qwen2.5:3b and Playwright on 127.0.0.1; 2 of 13 non-control archives caught (section 4). Same author, tiny n. Not a benchmark.

Not novel. Prior art covers the hidden-instruction canary page [26][27], the signed per-session token and the nonce round-trip [27], the coordination board's request that an agent disclose its task, tools and identity [29] [26], the decoy OpenAPI and llms.txt documents [27][28], the decoy MCP endpoint with a tools/list-then-call signal [28][30][31], invisible-link traps [35], asset starvation as a signal [27], timing [26], and, on the pipeline side, the hash-chained log [43], Isolation Forest [13], the coordination graph [6] and STIX export [22]. Drosera [27], Sundew [28], MCP Decoy [31] and the Thinkst products [29][30] are the nearest matches, and our honeypot side should be described as a reimplementation of these ideas plus extensions, not a new technique.

Where this stack extends prior art (possibly new, unproven). (1) Selective blocking versus starvation: the asset-starvation signal is typed by what was skipped, which removed the false positive for people who block images (in the agent-watch simulation, 15 of 15 such clients were alerted in v3 and 0 of 15 in v4). (2) Link-graph and trap evidence for browser-backed mimics: each page move is scored against the links the honeypot actually served, with an empirical fallback for ordinary logs, and an ARIA canary plus a three-boolean beacon target clients that look like browsers. In the 135-client pipeline simulation (9 mimic cohorts over 5 seeds) the share producing a lead rose from 0 percent in v2 to 33 percent in v3 and 78 percent in v4 (105 of 135). The cohort that loads CSS and script and sends correct headers (mimic_assets) went from 0 of 15 to 15 of 15, but only because the simulated mimic fetches our script and never runs it; two evaders (a stealth browser and a header forger) were still missed in that v4 simulation, 0 of 15 each. The v6 paired check moved those two cohorts; see the paragraph after the cost note, and do not read the v4 share as a current accuracy. (3) Coordination and harness evidence: a handoff token replayed from a different session or address, a capped claim loop, and a deterministic harness-archetype label with reasons. (4) Integration and honest reporting: the pseudonymized output feeds a separate pipeline with a two-signal corroboration gate and per-layer attribution of every lead. The search found each half in separate projects but no project doing both; finding no equivalent is not proof that none exists.

What the extensions cost. The beacon rule produces false positives on people whose script is blocked: 15 of 15 simulated beacon-blocking humans and 10 of 10 no-asset, no-Sec-Fetch humans received a weak lead. The corroboration gate removed 11 statistical-only benign leads (7 compliance scanners, 4 noisy-denied clients) in the standard scenario set and lowered the false-positive rate from 0.06 to 0.04, at the price of 2 injected probers and, in the all-evasive set, all recall that depended on Isolation Forest and the sequence model alone (core recall 0.24 to 0.00). Both trade-offs are reported in the project READMEs.

v6 synthetic check, still same-author [46]. On paired traffic, agent-watch human alerts fell from 15 of 75 to 0 of 75, the pipeline web false-positive rate from 39 of 575 (0.068) to 14 of 575 (0.024), and the core standard false-positive rate stayed 21 of 605 (0.035). All-evasive core recall rose from 0 of 80 to 12 of 80, all of it the access-prober cohort. In that same check the stealth browser and the header forger are no longer 0 of 15 pipeline leads (10 of 15 and 10 of 15). The cost is the documented regression: mimic_assets and mimic_linked_nosec fell from 15 of 15 to 0 of 15 pipeline leads, because they now look like the humans whose false positives were removed. Those figures do not describe the loopback sample in section 4, where the catch rate on non-control archives is 2 of 13.

Honest evaluation boundaries. The figures in this section, and in the v3 to v5 prototype table, are synthetic same-author regression checks, not benchmarks: the simulated clients, including each mimic, were written by the same author as the detectors, with the rules in mind. A honeypot or beacon contact shows contact, not intent. Every web-honeypot output is a lead for human review. The honeypot does not auto-block a public visitor. The lab gateway in section 6 can freeze only a registered internal key, and only inside that prototype. Palisade reports about 8.1 million real attempts [26] and the Li et al. honeysites 26.4 million requests [41]. This project has no wild deployment. The only runs against a live model are loopback: qwen2.5:3b and Playwright Chromium on a honeypot bound to 127.0.0.1, with a catch rate of 2 of 13 on the archived non-control sample (section 4). A second, larger local model was not run (about 5 GiB free RAM and no swap on that machine). There is still no TLS fingerprint layer, although measurement studies find multi-layer fingerprints strongly discriminative [38][39] and some agents bypassed every defense tested [39].

10. Limitations of this paper

- The sources were gathered by web search; a few primary documents were read in full (the EDPB draft guidelines, the arXiv paper on LLM bot detectors, and the ATLAS v2026.09 data file). Others rest on abstracts or summaries and should be checked before being cited onward.
- Several 2026 items are recent and may change, including the EDPB draft guidelines, ATLAS v2026.09 and the EU Digital Omnibus changes.
- Company reports (Anthropic, Meta) are self-reported.
- Prototype results are on synthetic data planted by the authors and do not measure real-world performance.
- The related-work search in section 9 was a limited public web search on 23 October 2026, not a systematic review; for GitHub projects the README was read, not the code, and none was run. Where a capability is marked “not found in my search” it may still exist. Several projects are weeks or months old and may change. The v3 to v5 figures in that section come from simulated clients written by the same author as the detectors.
- v3 to v6 synthetic figures come from simulated clients written by the same author as the detectors. The v6 paired check is in section 4 and in the project file docs/v6-changes.md [46]. It includes two documented pipeline regressions (mimic_assets and mimic_linked_nosec, each 15 of 15 to 0 of 15).

- The Tier-2 loopback sample is 14 archived runs on one machine and one 3B model. The 2 of 13 catch rate excludes the scripted human control. It must not be pooled with synthetic recall. No wild honeypot was deployed. A mid-size second model was not run (about 5 GiB free RAM, no swap).
- Honeypot or beacon contact is not intent. The web honeypot does not auto-block a visitor. Gateway freeze applies only to registered internal lab keys, on 127.0.0.1 by default. The Phase 2 and Phase 3 test counts (14 of 14, including 7 of 7 falsification; 10 of 10 analyzer unit tests) are regression checks of those packages, not a field trial. The analyzer smoke counts are the four rows in section 4 and were not extended.

References

[1] MITRE ATLAS. https://atlas.mitre.org/ ; data file v2026.09: https://github.com/mitre-atlas/atlas-data/blob/main/dist/v6/ATLAS-2026.09.yaml

[2] NIST. AI Risk Management Framework (released 26 Jan 2023): https://www.nist.gov/itl/ai-risk-management-framework ; NIST AI 600-1, Generative AI Profile (26 Jul 2024): https://doi.org/10.6028/NIST.AI.600-1

[3] Greshake, K. et al. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173. https://arxiv.org/abs/2302.12173

[4] OWASP. Top 10 for LLM Applications 2025: LLM01 Prompt Injection https://genai.owasp.org/llmrisk/llm01-prompt-injection/ ; LLM06 Excessive Agency https://genai.owasp.org/llmrisk/llm062025-excessive-agency/

[5] OWASP. Top 10 for Agentic Applications. https://genai.owasp.org/2025/12/09/owasp-top-10-for-agentic-applications-the-benchmark-for-agentic-security-in-the-age-of-autonomous-ai/

[6] Pacheco, D. et al. Uncovering Coordinated Networks on Social Media: Methods and Case Studies. ICWSM 2021. https://doi.org/10.1609/icwsm.v15i1.18075

[7] MITRE ATLAS technique entries AML.T0051 (LLM Prompt Injection), AML.T0053 (AI Agent Tool Invocation), AML.T0080 (AI Agent Context Poisoning), AML.T0086 (Exfiltration via AI Agent Tool Invocation) and AML.T0118 (Autonomous AI Agent Communication), data file v2026.09, as [1].

[8] Davis, C. et al. BotOrNot (2016) and Botometer versions, as described at https://github.com/osome-iu/botometer-python

[9] Yang, K.-C., Varol, O., Hui, P.-M., Menczer, F. Scalable and Generalizable Social Bot Detection through Data Selection. AAAI 2020. https://ojs.aaai.org/index.php/AAAI/article/view/5460/5316

[10] Orenstein, N., Birman, Y. Breaking and Defending LLM-Powered Social Media Bot Detection Systems. arXiv:2608.15893 (16 Aug 2026). https://arxiv.org/abs/2608.15893

[11] Feng, S. et al. TwiBot-22: Towards Graph-Based Twitter Bot Detection. NeurIPS 2022 Datasets and Benchmarks. https://arxiv.org/abs/2206.04564

[12] Chan, A. et al. Visibility into AI Agents. ACM FAccT 2024. https://arxiv.org/abs/2401.13138

[13] Liu, F. T., Ting, K. M., Zhou, Z.-H. Isolation Forest. IEEE ICDM 2008. https://doi.org/10.1109/icdm.2008.17

[14] Van Buren v. United States, 593 U.S. ___ (2021). https://www.law.cornell.edu/supremecourt/text/19-783

[15] hiQ Labs, Inc. v. LinkedIn Corp., 31 F.4th 1180 (9th Cir. 2022). https://cdn.ca9.uscourts.gov/datastore/opinions/2022/04/18/17-16783.pdf

[16] Meta Platforms, Inc. v. Bright Data Ltd. (N.D. Cal., summary judgment 23 Jan 2024), as reported at https://arstechnica.com/tech-policy/2024/01/facebook-suffers-big-loss-in-lawsuit-against-data-scraping-company/

[17] EDPB. Guidelines 03/2026 on web scraping in the context of generative AI, version 1.0 adopted 7 July 2026 for public consultation. https://www.edpb.europa.eu/system/files/2026-07/edpb_guidelines_2020603_webscraping_v1_en_0.pdf. Also: Digital Services Act Article 40 and Delegated Regulation (EU) 2025/2050 on researcher data access: https://digital-strategy.ec.europa.eu/en/news/commission-adopts-delegated-act-data-access-under-digital-services-act

[18] Meta. Inauthentic Behavior policy (coordinated inauthentic behavior). https://transparency.meta.com/policies/community-standards/inauthentic-behavior/

[19] Motwani, S. R. et al. Secret Collusion among AI Agents: Multi-Agent Deception via Steganography. NeurIPS 2024. https://arxiv.org/abs/2402.07510

[20] NIST. Technical Blog: Strengthening AI Agent Hijacking Evaluations (Jan 2025). https://www.nist.gov/news-events/news/2025/01/technical-blog-strengthening-ai-agent-hijacking-evaluations ; NIST AI 100-2 E2025: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-2e2025.pdf

[21] Anthropic. Disrupting an AI-orchestrated cyber espionage campaign. https://www.anthropic.com/news/disrupting-AI-espionage

[22] OASIS. STIX 2.1 https://www.oasis-open.org/standard/stix2-1/ ; TAXII 2.1 https://www.oasis-open.org/standard/taxii2-1/ ; MISP https://www.misp-project.org/

[23] ISO/IEC 29147:2018 https://www.iso.org/standard/72311.html ; CERT Guide to Coordinated Vulnerability Disclosure https://certcc.github.io/CERT-Guide-to-CVD/ ; CISA CVD program https://www.cisa.gov/resources-tools/programs/coordinated-vulnerability-disclosure-cvd-program

[24] EU AI Act, Article 73 (serious incidents) https://www.euaiact.com/article/73 ; implementation timeline (Commission AI Act Service Desk) https://ai-act-service-desk.ec.europa.eu/en/ai-act/timeline/timeline-implementation-eu-ai-act

[25] The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research (DHS, 2012). https://www.dhs.gov/sites/default/files/publications/CSD-MenloPrinciplesCORE-20120803_1.pdf

[26] Reworr, Volkov, D. LLM Agent Honeypot: Monitoring AI Hacking Agents in the Wild. arXiv:2410.13919 (17 Oct 2024, rev. 10 Feb 2025). https://arxiv.org/abs/2410.13919 ; code https://github.com/PalisadeResearch/llm-honeypot

[27] Drosera, a honeypot for autonomous AI agents (rod-trent), PyPI 0.1.0 on 5 Sep 2026. https://pypi.org/project/drosera/ ; https://github.com/rod-trent/Drosera

[28] Sundew, a honeypot for AI agents (sundew-sh), PyPI 0.1.0 on 25 Feb 2026. https://pypi.org/project/sundewsh/ ; https://docs.sundew.sh/

[29] Thinkst. Getting Agents to tell on themselves (Agent Provocateur), 4 Sep 2026. https://blog.thinkst.com/2026/09/getting-agents-to-tell-on-themselves.html ; https://help.canary.tools/hc/en-gb/articles/38888479188637-How-do-I-configure-my-Canary-to-detect-interactions-from-AI-agents

[30] Thinkst. MCP Configuration Canarytoken, 22 Jun 2026. https://docs.canarytokens.org/guide/mcp-token.html ; https://blog.thinkst.com/2026/06/one-for-all-the-models-out-there.html

[31] gweber. MCP Decoy Server (repository created 22 Apr 2026). https://github.com/gweber/mcp-decoy

[32] barvhaim. HoneyMCP (PyPI 0.1.8, 6 Aug 2026). https://pypi.org/project/honeymcp/ ; https://github.com/barvhaim/HoneyMCP

[33] Beelzebub Labs. Beelzebub deception runtime (MCP deception service). https://github.com/beelzebub-labs/beelzebub

[34] Cloudflare. Web Bot Auth documentation (updated 8 Oct 2026): https://developers.cloudflare.com/bots/reference/bot-verification/web-bot-auth/ ; Meunier, T., Galicer, M. Forget IPs: using cryptography to verify bot and agent traffic (15 May 2025): https://blog.cloudflare.com/web-bot-auth/

[35] Cloudflare. Trapping misbehaving bots in an AI Labyrinth (19 Mar 2025). https://blog.cloudflare.com/ai-labyrinth/

[36] DataDome. AI Agent Identification and Intent-based Protection documentation. https://docs.datadome.co/docs/ai-agent-identification ; https://docs.datadome.co/docs/intent-based-protection

[37] FoxIO. JA4+ network fingerprinting. https://github.com/FoxIO-LLC/ja4 ; Cloudflare. Advancing Threat Intelligence: JA4 fingerprints and inter-request signals (12 Aug 2024). https://blog.cloudflare.com/ja4-signals/

[38] Wang, E., Shafiq, Z., Vekaria, Y. FP-Agent: Fingerprinting AI Browsing Agents. arXiv:2605.01247 (2 May 2026). https://arxiv.org/abs/2605.01247

[39] Fayolle, I. et al. On the Internet, Nobody Knows You're an LLM Bot: Unmasking Web Agents with Multi-Layer Fingerprinting. arXiv:2606.30119 (29 Jun 2026). https://arxiv.org/abs/2606.30119

[40] Lugoloobi, W. et al. Known By Their Actions: Fingerprinting LLM Browser Agents via UI Traces. arXiv:2605.14786 (14 May 2026). https://arxiv.org/abs/2605.14786

[41] Li, X., Azad, B. A., Rahmati, A., Nikiforakis, N. Good Bot, Bad Bot: Characterizing Automated Browsing Activity. IEEE S&P 2021. https://doi.org/10.1109/sp40001.2021.00079 ; poster https://www.ieee-security.org/TC/SP2021/downloads/poster/poster15.pdf

[42] Cowrie https://github.com/cowrie/cowrie ; T-Pot https://github.com/telekom-security/tpotce ; OpenCanary https://github.com/thinkst/opencanary

[43] Schneier, B., Kelsey, J. Secure audit logs to support computer forensics. ACM TISSEC 1(3), 1999. https://www.schneier.com/academic/archives/1999/05/secure_audit_logs_to.html

[44] Doran, D., Gokhale, S. S. Web robot detection techniques: overview and limitations. Data Mining and Knowledge Discovery 22, 2011. https://doi.org/10.1007/s10618-010-0180-z

[45] Project file docs/prior-art.md (prior-art table with dates, verification notes and unverified items), 23 Oct 2026.

[46] Project files for this update, 23 to 24 October 2026: docs/v6-changes.md (paired synthetic v5/v6 counts); docs/phase1-tier2-report.md and docs/phase1-fixes.md (loopback archive, R14 and dwell calibration); docs/phase2-gateway.md (AML.M0030 lab gateway, 14 of 14 tests); docs/phase3-analyzer.md (offline analyzer, 12 Sigma rules, 10 of 10 unit tests). The 2 of 13 catch rate and the four-row analyzer smoke are stated in section 4. The smoke was run on Jordan's machine against archived Tier-2 logs and agentwatch-analyzer/tests/fixtures/agentlike_nginx.log.

[47] Project files for the 25 October 2026 update: agent-gateway/agent_gateway/collusion_eval.py and sprint_submission/ (collusion_sprint_report.pdf, collusion_sprint_slides.pdf, collusion_benchmark_results.json), generated at commit cefec0d by python -m agent_gateway evaluate-collusion --report.